

4.4 Making Third party API calls with RestClient

• How does Spring call external APIs ?

- There are **3 ways**:
 - RestTemplate (old)
 - RestClient (new sync)
 - WebClient (modern async/reactive)

• RestTemplate (Old Way):

- RestTemplate is a **synchronous HTTP client** used to call external APIs.

```
RestTemplate restTemplate = new RestTemplate();
String response = restTemplate.getForObject(
    "https://api.example.com/users", String.class
);
```

- **How it works:** Request → waits → response comes → returns
- **Problems:**
 - Blocking (bad for scalability)
 - Hard to customize
 - Old design
- **Status:** Deprecated (Spring recommends not using it for new projects)

• RestClient (Modern Replacement):

- RestClient is the new synchronous HTTP client introduced in Spring 6.

```
RestClient restClient = RestClient.create();
String response = restClient.get()
    .uri("https://api.example.com/users")
    .retrieve()
    .body(String.class);
```

- **Features:**
 - Cleaner API
 - Fluent style (builder pattern)
 - Better error handling
 - Still synchronous (blocking)
- **Flow:** Request → wait → response → return
- Same as RestTemplate but better design.
- RestClient is a modern, fluent, synchronous HTTP client replacing RestTemplate.

• WebClient (Advanced):

- WebClient is a **non-blocking**, reactive HTTP client.

```
WebClient webClient = WebClient.create();
Mono<String> response = webClient.get()
    .uri("https://api.example.com/users")
    .retrieve()
    .bodyToMono(String.class);
```

- **Flow:** Request → don't wait → continue → response later
- **Returns:**
 - Mono<T> → single value
 - Flux<T> → multiple values

4.4 Making Third party API calls with RestClient

- **Advantages:**
 - High performance
 - Handles many concurrent calls
 - Async + reactive
- **Complexity:**
 - Harder to understand
 - Reactive programming needed

• Step-by-step: Setting up RestClient correctly (Standard Way)

• Configure RestClient:

```
@Configuration
public class RestClientConfig {

    @Value("${collegeManagementSystem.base.url}")
    private String BASE_URL;

    @Bean
    @Qualifier("collegeManagementSystemRestClient")
    public RestClient getCollegeManagementSystemRestClient() {
        return RestClient.builder()
            .baseUrl(BASE_URL)
            .defaultHeader(CONTENT_TYPE, APPLICATION_JSON_VALUE)
            .build();
    }
}
```

• Create DTOs (must be serializable)

```
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
@Builder
public class StudentRequestDto {
    private String name;
    private String registrationNumber;
    private List<Long> subjectIds = new ArrayList<>();
    private Integer fees;
}
```

• Inject RestClient into your client class

```
@Service
@RequiredArgsConstructor
public class StudentClientImpl implements StudentClient {

    @Qualifier("collegeManagementSystemRestClient")
    private final RestClient restClient;
}
```

- Without @Qualifier, wrong bean may be injected

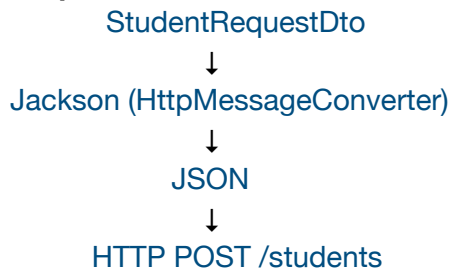
• Make API calls

```
public StudentResponseDto createStudent(StudentRequestDto requestDto) {
    ApiResponse<StudentResponseDto> response = restClient.post()
        .uri("students")
        .retrieve()
        .body(new ParameterizedTypeReference<>() {});
    return response.getData();
}
```

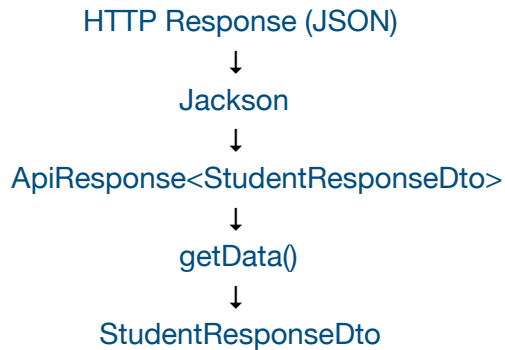
4.4 Making Third party API calls with RestClient

- What happens internally

- Request flow



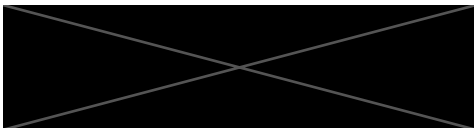
- Response flow



- Expected API response format

- Your backend must return:

```
{  
  "data": {  
    "id": 1,  
    "name": "Raj Prasad Swain",  
    "registrationNumber": "2301326154"  
  }  
}
```



- Errors in RestClient:

- When you call a third-party API, Things can go wrong:
 - API returns **404** (Not Found)
 - API returns **500** (Server Error)
 - Network failure
 - Timeout
- So, Error handling = how you detect and handle these failures properly
- By default, RestClient: **4xx** / **5xx** → throws exception
- Example:
 - **404** → HttpClientErrorException
 - **500** → HttpServerErrorException

4.4 Making Third party API calls with RestClient

• Errors Handling in RestClient:

• Basic Error Handling (Try-Catch):

```
try {
    ApiResponse<List<StudentResponseDto>> students = restClient.get()
        .uri("students")
        .retrieve()
        .body(new ParameterizedTypeReference<>() {
        });
    return students.getData();
} catch (Exception e) {
    throw new RuntimeException("Failed to retrieve students: ", e);
}
```

• Problem

- Too generic
- Not clean
- Not scalable

• Standard Way, Handle per request: **onStatus()**

```
ApiResponse<StudentResponseDto> apiResponse = restClient.post()
    .uri("students")
    .body(requestDto)
    .retrieve()
    .onStatus(HttpStatusCode::is4xxClientError, (request, response) -> {
        throw new RuntimeException("Client Error: " + response.getStatusCode());
    })
    .onStatus(HttpStatusCode::is5xxServerError, (request, response) -> {
        throw new RuntimeException("Server Error: " + response.getStatusCode());
    })
    .body(new ParameterizedTypeReference<>() {});
```

- If 4xx → custom handling
- If 5xx → custom handling
- Else → success

• Standard Way, Handle globally: **defaultStatusHandler()**

```
@Bean
@Qualifier("collegeManagementSystemRestClient")
public RestClient getCollegeManagementSystemRestClient() {
    return RestClient.builder()
        .baseUrl(BASE_URL)
        .defaultHeader(CONTENT_TYPE, APPLICATION_JSON_VALUE)
        .defaultStatusHandler(HttpStatusCode::is5xxServerError, (request, response) -> {
            throw new RuntimeException("Server Error: " + response.getStatusCode());
        })
        .build();
}
```

- **onStatus()** handles errors per request, while **defaultStatusHandler()** defines global error handling for all requests created by that RestClient.
- RestClient provides **onStatus()** to handle HTTP errors like 4xx and 5xx. We map them to custom exceptions and handle them globally.